WEB DEVELOPMENT · 3RD SEMESTER

Week 06 · Block 6b

Events & Interactivity

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Respond to user actions with `addEventListener`
- Use the **event object**: `e.target`, `e.preventDefault()`
- Handle a **form submit** in JS — with **no page reload**
- Use **event delegation** for dynamic lists
- Build an **interactive to-do list**

In Block 6a you reshaped the page — but only once, on load.

Now the page reacts.

Clicks, typing, form submits — your JavaScript runs whenever the user does something.

PART 1

Events & Listeners

Events are things that happen

The browser fires an **event** for every user action — **click**, **input**, **submit**, **keydown**, **mouseover**...

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  console.log("clicked!");
});
```

- `addEventListener(type, handler)` — run code when that event fires
- You **listen** only for the events you care about

The handler is a callback

```
function onClick() {
  console.log("clicked!");
}

button.addEventListener("click", onClick);
```

- The handler is just a **function** — the first-class callback from Block 5a
- The browser calls it **for you**, every time the event happens
- Name it, or inline an arrow — both are fine; one element can have **many** listeners

PART 2

The Event Object

Every handler gets an event

```
button.addEventListener("click", (e) => {
  console.log(e.type);     // "click"
  console.log(e.target);   // the element that was clicked
});
```

- The handler receives an **event object**, by convention `e`
- **`e.target`** — the element that fired the event
- **`e.type`** — which event it was
- …plus mouse position, the key pressed, and more

preventDefault

Some events carry a **default browser action**:

- a form **submit** → reloads / navigates
- a link **click** → follows its `href`

```
link.addEventListener("click", (e) => {
  e.preventDefault();   // don't follow the link…
  // …do something in JavaScript instead
});
```

`e.preventDefault()` cancels that default, so *you* stay in control.

PART 3

Forms Without Reload

The classic reload problem

A `<form>` submit **reloads the page** — a whole new request/response *(Block 1)*. For an interactive app, you usually don't want that.

```
form.addEventListener("submit", (e) => {
  e.preventDefault();           // stop the reload
  const text = input.value;     // read the field (Block 2b)
  addTodo(text);                // do the work (your code)
  input.value = "";             // clear the field
});
```

Listen for **`submit`**, call **`preventDefault()`**, then handle it yourself.

Three technologies, one moment

That one handler ties the whole course together:

- `submit` fires on Enter or the button *(forms — Block 2b)*
- `preventDefault()` stops the page reload *(request/response — Block 1)*
- read `input.value`, then **update the DOM** *(Block 6a)*

This is the core loop of every interactive page: **user acts → JS runs → DOM updates**.

PART 4

Event Delegation

New elements have no listeners

- You add a listener to the buttons that exist **now**
- Items you create **later** — like new to-do items — won't have it
- Re-attaching listeners to every new element is tedious and bug-prone

There's a better way, and it uses how events travel.

Listen on the parent

```
list.addEventListener("click", (e) => {
  if (e.target.matches(".delete")) {
    e.target.closest("li").remove();
  }
});
```

- Put **one** listener on the parent (`<ul>`)
- Events **bubble up** from the clicked child to the parent
- Check **`e.target`** to see what was actually clicked
- Works for items added **later** — perfect for dynamic lists

PART 5

Putting It Together

The to-do list is all of today

- **submit** the form → `preventDefault`, read the text, **create** an `<li>`, **append** it *(Block 3 + Block 6a)*
- **click** an item → toggle a `done` class · **click** its delete button → remove it *(delegation)*
- clear the input after each add

Everything from Block 6, in one small app.

A cleaner shape (preview)

```
let todos = [];                          // the state
function render() { /* draw todos → DOM */ }

// add:    todos.push(text);                render();
// delete: todos = todos.filter(…);         render();
```

- Keep the **data** in an array; **render** it into the DOM
- Every change updates the array, then re-renders
- This **state → render** idea is the heart of frameworks — **Blocks 8 & 9**

Recap

- `addEventListener(type, handler)` runs a **callback** when an event fires
- The handler gets an **event object**: `e.target`, `e.type`, `e.preventDefault()`
- Handle a **form** in JS: `submit` → `preventDefault()` → read `.value` — **no reload**
- **Event delegation**: one listener on a parent handles its children, even future ones
- Events + DOM = **interactive** pages; keeping **state** and rendering it scales up *(Block 8)*

<EXERCISE> · 45 MIN

Your turn 👉

Build an **interactive to-do list** — all of Block 6 in one app.

1. **Add** items via the form: `submit` → `preventDefault` → create an `<li>` → `append`
2. **Clear** the input after each add (and ignore empty input)
3. **Mark done**: clicking an item toggles a `done` class *(delegation)*
4. **Delete**: each item has a delete button that removes it *(delegation)*

Starter: `exercises/week06/6b/` — open `README.md`. A form, an empty list, and a linked `app.js`.